プログラミング初学者の開発操作を支援する Visual Studio Code 拡張機能の設計・実装と予備的評価

― Quick Actions の開発を通じた操作性向上の検討 ―

著者：西田　英生

所属：大原情報医療保育専門学校和歌山校 情報処理学科 システム開発コース

執筆日：2026 年 11 月

概要

本研究では、プログラミング初学者の開発環境における操作を支援することを目的として、Visual Studio Code（以下、VS Code）の拡張機能「Quick Actions」の設計および開発を行った。

VS Code では、コード編集やファイル操作など、さまざまな機能が提供されている。一方、開発環境の操作に慣れていない利用者は、目的の機能や操作方法を把握するまでに時間を要する場合がある。特に、多数のキーボードショートカットを覚えることや、利用可能な機能の存在を把握することは、初学者にとって負担となる可能性がある。

そこで、本研究では、頻繁に使用する編集操作を GUI 上のボタンとして配置し、機能を視覚的に確認しながら利用できる拡張機能を開発した。また、C++、Python、Java の 3 言語に対応したコードテンプレート機能を実装し、基本構文からアルゴリズムまで、簡単な操作でコードを挿入できるようにした。さらに、不要な空白の削除や文字変換、行の上下移動など、日常的なコード編集を補助する機能を用意した。

本ツールは、開発者自身の作業効率を向上させる個人用ツールとして制作を開始したものである。同時に、VS Code の操作やプログラミングに慣れていない初学者でも、各機能の意味を把握しながら利用できることを設計方針とした。

本開発では、自然言語による指示を通じて生成 AI にコードの生成や修正を依頼し、動作を確認しながら改善を繰り返すバイブコーディングを開発手法として採用した。特に、制作における主目的を明確に提示するとともに、変更対象、変更禁止箇所、問題発生時のロールバック方針をプロンプトに指定し、既存機能への影響を抑えることを重視した。

開発過程では、生成 AI によるコード生成を合計 14 回実施し、開発者自身による対応を 4 回行った。その内訳は、旧バージョンへのロールバックが 3 回、コードの可読性を向上させるための修正が 1 回である。開発中には、プロンプトの設計に伴う意図しない UI 変更、インポート後に正常に使用できない不具合、ボタン操作ができなくなる問題なども発生した。

開発した拡張機能については、自動テストによる機能検証および第三者による簡易的な使用感の確認を実施した。使用者は 4 名であり、そのうち 3 名から回答を得た。使用感については、VS Code 標準のショートカットキーを多数覚える必要がなく、ボタンを 1 回クリックすることで目的の操作を実行できる点や、これまで知らなかった機能を発見して利用できる点について肯定的な意見が得られた。また、一括保存機能については、複数のファイルを扱う際に保存操作をまとめられる点が評価された。行の上下移動機能についても、HTML のナビゲーションなどの記述順序を変更する際に、簡単にコードの並び替えを行える点が利便性として挙げられた。一方で、機能の表示位置が分かりにくいこと、2 列配置によって画面が縦方向に長くなること、1 行単位での選択機能に対する要望など、改善すべき点も確認された。

本研究では、これらの開発経験と使用感に関する意見を整理し、初学者向け開発支援ツールにおける操作性、既存機能との差別化、および生成 AI を利用した開発管理について考察する。

キーワード：Visual Studio Code、拡張機能、GUI、プログラミング初学者、バイブコーディング、生成 AI、開発支援、プロンプト設計、バージョン管理

第 1 章 はじめに

1.1 研究背景

プログラミングを行う際には、ソースコードの記述だけでなく、ファイルの保存、文字列の編集、検索、インデントの調整など、さまざまな操作が必要となる。

これらは開発作業の基本となる操作である。しかし、開発環境の操作に慣れていない利用者にとっては、目的の機能がどこに配置されているのか、また、どのような方法で実行できるのかを把握することが難しい場合がある。

特に、キーボードショートカットやコマンドパレットの操作方法を十分に理解していない場合、目的の操作を実行するために、機能の場所や使用方法を調べる必要がある。

VS Code には、さまざまな操作に対応した標準ショートカットキーが用意されている。一方、多数のキーの組み合わせを覚えることは、初学者にとって負担となる可能性がある。また、便利な機能が用意されていても、その機能やショートカットキーの存在を知らなければ、十分に活用できない場合がある。

このような課題に対して、開発環境の操作を視覚的に提示し、利用者が画面上から目的の機能を選択できる仕組みは、操作方法の習得を補助する一つの方法になると考えられる。

また、近年では生成 AI を利用したコード生成や修正が可能になり、個人でもソフトウェア開発を進めやすい環境が整いつつある。一方、生成 AI による意図しないコード変更や、既存機能に影響を与える修正など、利用者側で管理すべき課題も存在する。

そのため、生成 AI を利用した開発では、機能の実装だけでなく、指示内容の管理、変更範囲の限定、動作確認、問題発生時の復旧方法についても考慮する必要がある。

1.2 研究目的

本研究では、プログラミング初学者が VS Code の基本的な開発操作を簡単に利用できるようにすることを目的として、拡張機能 Quick Actions を設計・開発する。

本ツールは、開発者自身の作業効率を向上させる個人用ツールとして制作を開始したが、同時に、VS Code の操作やプログラミングに慣れていない初学者でも利用しやすいことを意識して設計した。

具体的には、以下の 3 点を研究上の目標とする。

1. 頻繁に使用する開発操作をボタンとして配置し、機能を視覚的に把握できるようにする。
2. 不要な空白の削除や文字変換などの編集操作を簡略化する。
3. プログラミング言語ごとのコードテンプレートを提供し、構文の記述や利用を支援する。

さらに、標準ショートカットキーを利用する場合と、Quick Actions のボタンから操作する場合の使用感についても意見を収集する。

開発した拡張機能を第三者に使用してもらい、操作性や利便性、改善点について意見を収集するとともに、生成 AI を利用した開発過程で発生した問題と、その対策について整理する。

1.3 本研究の位置づけ

本研究は、新しいプログラミング言語やアルゴリズムの開発を目的とするものではなく、既存の開発環境における操作を支援する拡張機能を対象とする。

特に、単に操作機能を追加するのではなく、開発環境に慣れていない利用者が必要な機能を認識し、簡単かつ安全に利用できることを重視する。

また、本ツールは開発者自身が利用することを出発点とした個人開発であり、一般利用者に対する有効性を実証することを目的としたものではない。

開発を通じて得られた問題や改善経験を整理し、生成 AI を活用した個人開発における操作性と開発管理の課題を検討する実践的な取り組みとして位置づける。

第 2 章 関連技術および設計上の考え方

2.1 GUI による操作支援

GUI（Graphical User Interface）は、画面上のボタンやメニューなどの視覚的要素を通じてコンピューターを操作する仕組みである。

Shneiderman は、直接操作に関する研究において、対象を視覚的に提示し、複雑なコマンド構文に依存せず操作できるインターフェースの特徴について論じている[1]。

本研究では、この考え方を参考に、VS Code の基本操作をボタンとして提示する設計を採用した。これにより、利用者がコマンド名やキーボードショートカットを記憶していなくても、画面上から目的の機能を選択できる構成を目指した。

2.2 ユーザビリティに関する考え方

Nielsen は、ユーザーインターフェースの設計原則として、システム状態の可視性、エラーの防止、記憶よりも認識を重視する設計などを挙げている[2]。

これらの原則は、初学者向け開発支援ツールの設計を検討するうえでも参考となる。

例えば、機能名をボタンに表示することにより、利用者が操作方法を記憶していなくても、機能の内容を視覚的に把握しやすくなる。また、処理の完了や失敗を画面に表示することで、操作結果を確認しやすくなる。

本研究では、これらの考え方を参考に、機能名の表示、操作結果の通知、必要な操作の実行前確認などを実装した。

また、標準ショートカットキーを覚えていなくても画面上から操作できる点についても、記憶への依存を減らすための設計上の特徴として位置づける。

2.3 Visual Studio Code の拡張機能

VS Code は、Extension API を通じてエディターの機能やユーザーインターフェースを拡張できる開発環境である[3]。

また、Webview を利用することで、HTML、CSS、JavaScript を用いた独自の画面を構築できる。Webview と拡張機能本体は、メッセージ通信によって連携できる[4]。

本研究では、これらの仕組みを利用し、VS Code 上で動作する独自の操作パネルを実装した。

2.4 バイブコーディングと生成 AI

自然言語による指示を通じて生成 AI にプログラムコードを生成させ、実行結果を確認しながら修正を依頼する開発方法は、一般にバイブコーディングと呼ばれる[5]。

本研究では、生成 AI との対話を通じて機能の実装や修正を進める開発方法を、バイブコーディングとして扱う。

ただし、本研究では生成 AI にすべての判断を任せたわけではない。開発者が制作目的や機能要件を設定し、変更範囲を指定したうえで、生成されたコードの動作確認や修正内容の判断を行った。

そのため、本研究におけるバイブコーディングは、生成 AI を利用しながら開発者が仕様や品質を管理する協働型の開発手法として位置づける。

2.5 既存機能・拡張機能との比較

VS Code には、開発作業を効率化するための標準機能や、多数の拡張機能が提供されている。

標準ショートカットキーでは、保存、検索、編集などの操作をキーボードから実行できる。また、標準スニペットでは、あらかじめ登録されたコードを補完候補や Tab 操作によって挿入できる[7][8]。

さらに、複数のコマンドを連続実行する Multi-command[9]や、VS Code のコマンドをボタンから実行できる Quick Command Toolbar[10]などの拡張機能も存在する。

これらの既存機能・拡張機能と Quick Actions の主な特徴を比較する。

比較対象	主な特徴	Quick Actions との違い
VS Code 標準ショートカット	キーの組み合わせによる編集・保存・検索	キー操作を覚えず、画面上のボタンから実行できる
VS Code 標準スニペット	登録したコードを補完候補や Tab 操作で挿入	言語別テンプレートを専用画面で選択し、説明やプレビューを確認できる
Multi-command	複数のコマンドを一つのコマンドとして実行	初学者が使用する基本操作を、あらかじめボタンとしてまとめている
Quick Command Toolbar	VS Code のコマンドをボタンから実行	コード編集支援機能と複数言語のテンプレートを統合している

既存の拡張機能にも Quick Actions と共通する機能が存在するため、ボタン操作やコード挿入そのものが本ツール独自の技術であるとはいえない。

Quick Actions の特徴は、基本的な編集操作、コード整理・変換機能、C++・Python・Java のコードテンプレートを一つの操作パネルにまとめている点にある。

また、初学者の利用を意識し、機能名を視覚的に提示するとともに、テンプレートの説明やプレビューを確認してからコードを挿入できる構成とした。

以上から、Quick Actions は既存の拡張機能を置き換えることを目的とするものではなく、複数の機能を一つの画面にまとめ、VS Code の操作に不慣れな利用者が必要な機能を発見し、利用できるようにする補助的なツールとして位置づける。

なお、この比較は公開されている機能説明に基づくものであり、すべての拡張機能を実際にインストールして性能比較を行ったものではない。

第 3 章 システムの設計

3.1 システム概要

本研究で開発した Quick Actions は、VS Code 上で利用する開発支援拡張機能である。

基本的な設計方針は、開発時によく使用する操作を GUI 上にまとめ、利用者がボタンを選択することで各種操作を実行できるようにすることである。

また、機能を「操作」と「テンプレート」の 2 つに分類し、通常の編集作業とプログラム構文の挿入を区別できる構成とした。

本ツールの設計では、開発者自身の利便性と初学者の利用しやすさを両立させることを重視した。単に多くの機能を搭載するのではなく、操作の目的を理解しやすくし、少ない手順で目的を達成できることを目指した。

3.2 ユーザーインターフェースの設計

Quick Actions の操作パネルは、VS Code の右側にある Secondary Side Bar に配置した。

当初、Quick Actions を左側の画面に配置すると、ファイルやフォルダーの構成を確認する Explorer と表示領域が競合する問題があった。

この問題に対応するため、右側の Secondary Side Bar にも配置できる構成とした。これにより、左側で Explorer を利用しながら、右側で Quick Actions を操作できるようにした。

図 1 に Quick Actions のメイン画面を示す。画面右側の操作パネルに「操作」と「テンプレート」のタブを設け、操作機能をボタンとして一覧できる構成とした。ボタンには機能名を表示し、利用者がコマンド名やショートカットキーを記憶していなくても、画面上から目的の操作を選択できるようにした。

- 操作とテンプレートを切り替えるタブ
- 操作やテンプレートを検索する検索欄
- 機能ごとに分類された操作ボタン
- C++、Python、Java を切り替える言語タブ
- コードテンプレートの一覧
- テンプレートの説明およびプレビュー
- 操作結果を表示するステータス領域

操作ボタンは基本的に 2 列で配置し、限られた画面内で複数の機能を確認できる構成とした。また、機能を分類して表示することで、利用者が目的に応じて操作を探しやすくすることを意図した。

ただし、2 列配置については、実際に使用した利用者から、画面が縦方向に長くなり、使いにくいという意見も得られている。この点については、第 5 章および第 6 章で検討する。

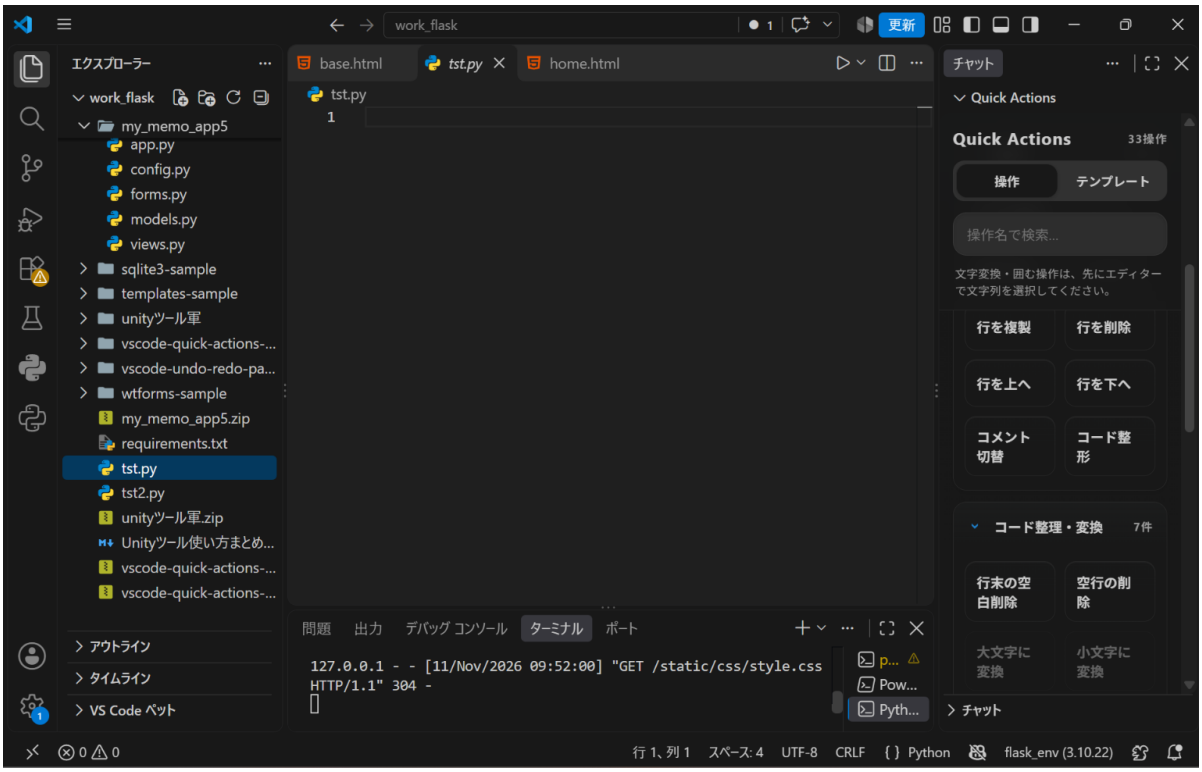


図 1　Quick Actions のメイン画面

3.3 機能設計

Quick Actions には、基本的な編集操作、画面操作、コード整理・変換などの機能を実装した。主な機能を以下に示す。

分類	主な機能
基本操作	元に戻す、やり直す、保存、すべて保存
編集操作	コピー、貼り付け、切り取り、行の複製、行の削除、行の上下移動
検索・移動	ファイル内検索、検索と置換、行へ移動
画面操作	拡大、縮小、表示リセット、下部パネル表示切り替え
コード整理	行末の不要な空白削除、空行の削除
文字変換	選択文字列の大文字・小文字変換
編集補助	インデントの追加・削除、括弧・引用符による文字列の囲み、コメント化

これらの機能をボタンとして配置し、コマンドの入力やショートカットキーの確認を行わずに操作できる構成とした。

特に、行末の空白削除や空行の削除など、コードの整理に関する操作を個別の機能として用意した。

また、コメント化、括弧による囲み、文字の大文字・小文字変換、行の上下移動など、プログラミング中に繰り返し発生する細かな編集作業を簡単に実行できるようにした。

3.4 コードテンプレートの設計

コードテンプレート機能では、C++、Python、Java の 3 言語に対応した。

各言語につき 20 種類、合計 60 種類のテンプレートを収録した。

テンプレートは、基本構文から応用的なアルゴリズムへと順番に配置し、難易度ラベルや番号による分類は行わない構成とした。

主な収録内容は以下のとおりである。

- 基本構文：if、for、while、関数、クラス
- データ構造：配列、vector、list、map、辞書
- 探索・ソート：線形探索、ソート、二分探索
- グリッド：4 方向探索、グリッド BFS
- グラフ：幅優先探索、深さ優先探索
- アルゴリズム：累積和、1 次元 DP、Union-Find、ダイクストラ法

各テンプレートには、説明、挿入位置の目安、必要な準備、コード本文を設定した。

図 2 にコードテンプレート機能の画面を示す。画面上部で C++、Python、Java の言語を切り替えられ、テンプレートはカード形式で一覧表示される。各カードには用途の説明を付け、利用者がテンプレートの目的を確認してから選択できるようにした。

また、現在開いているファイルの言語を自動判別するほか、言語タブから手動で変更することもできる。テンプレートを選択するとコードのプレビューが表示され、利用者は内容を確認したうえで挿入できる。挿入時には Tab キーで入力箇所を移動できるスニペット形式を採用した。

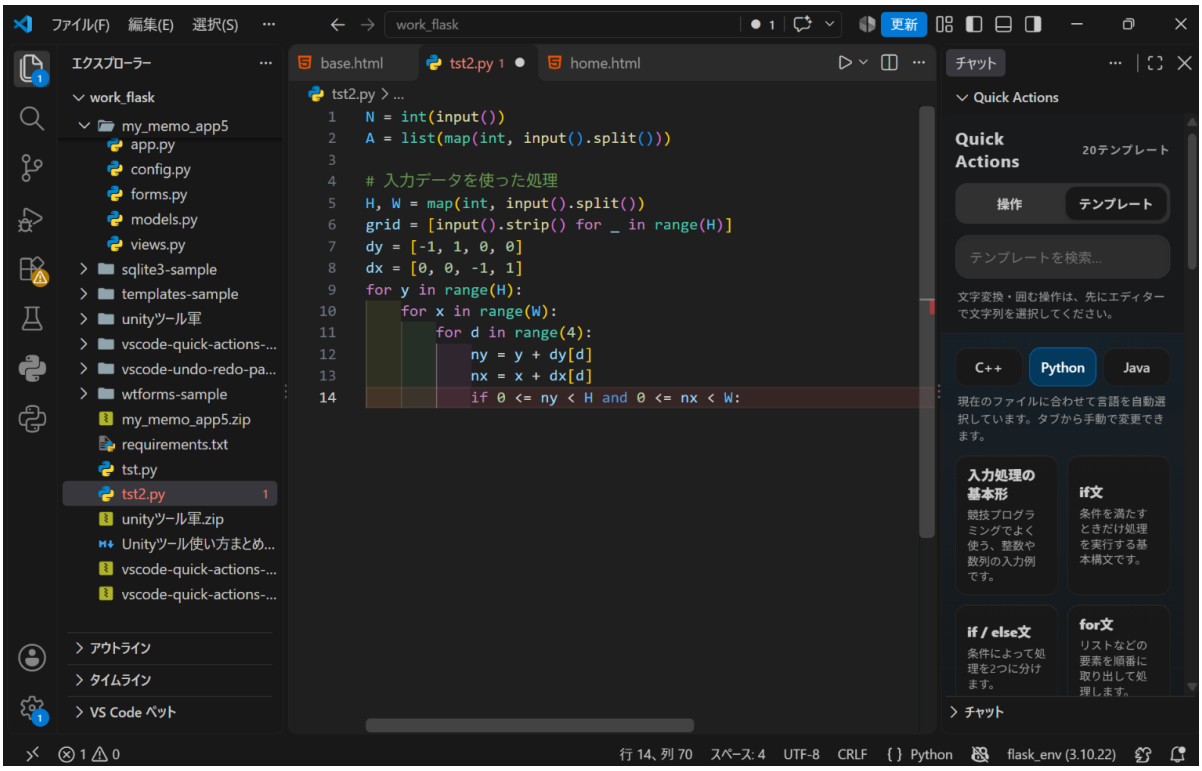


図 2　コードテンプレート機能の画面

第 4 章 システムの実装

4.1 開発環境

本研究では、VS Code の拡張機能として Quick Actions を開発した。主な構成技術を以下に示す。

項目	開発環境・使用技術
開発環境	Visual Studio Code
拡張機能本体	JavaScript
ユーザーインターフェース	HTML、CSS、JavaScript
テンプレート管理	JSON
VS Code 連携	VS Code Extension API
AI 支援	ChatGPT

開発した拡張機能の対象バージョンは 1.7.0 であり、VS Code 1.106.0 以降を対象とする。

4.2 バイブコーディングを用いた開発手法

本研究では、生成 AI を利用してコードの生成や修正を行い、実際の動作確認を繰り返しながら拡張機能を開発した。この開発手法として、バイブコーディングを採用した。

開発者は、実現したい機能、改善したい箇所、画面構成などを自然言語によって生成 AI に提示した。生成 AI は、その指示に基づいてソースコードの生成や既存コードの修正案を提示し、開発を支援した。

本開発において重視したのは、生成 AI に単にコードの作成を依頼することではなく、制作の主目的を一貫して提示し、その目的から逸脱しないように開発を進めることである。

本研究における最優先の目的は、次のとおりである。

「プログラミング初学者が開発作業で困難に感じる操作を、機能の意味を理解しながら、簡単かつ安全に実行できるよう支援すること。」

この目的を開発の各段階で生成 AI に提示し、機能追加や画面変更を行う際にも、初学者の開発作業を支援するという本来の方向性を維持することを重視した。

4.2.1 開発者と生成 AI の役割分担

本開発における役割分担を以下に示す。

担当	主な内容
開発者	研究目的の設定、機能要件の決定、画面構成の検討、変更対象の指定、変更禁止箇所の指定、動作確認、採用する変更内容の判断、旧バージョンへの復元
生成 AI	ソースコードの生成、既存コードの修正案の提示、実装方法の提案、エラー原因の調査支援、技術的な説明

このように、開発者が仕様や方向性を決定し、生成 AI が実装を支援する形式で開発を進めた。

特に、AI による修正をそのまま採用するのではなく、最終的な判断や動作確認、問題が発生した場合の復旧作業は開発者自身が行うことを重視した。

4.2.2 プロンプトによる変更範囲の管理

生成 AI を利用した開発では、指示内容によって生成されるコードや修正範囲が変化する可能性がある。

また、チャット形式で入力した指示は、会話が長期化した場合や別の作業へ移行した場合に、以前の条件が必ずしも十分に反映されるとは限らない。

そのため、本研究では、継続的に適用する共通の指示（以下、共通プロンプト）と、個別の修正作業に関する指示（以下、個別プロンプト）を分けて設定する方法を採用した。

共通プロンプトには、開発全体で継続的に適用したい内容を設定する。具体的には、以下のような内容である。

1. 改善案や提案を求められた場合は、原則として 3 つ程度の案を提示する。
2. 情報の正確性を確認できるよう、根拠となる資料や公式情報へのリンクを提示する。
3. 制作の主目的から逸脱しない。
4. 既存機能や操作方法を不用意に変更しない。
5. 指定されていない機能を勝手に削除・変更しない。

6. 不具合が発生した場合に備え、変更前の状態へ戻せることを考慮する。

一方、個別プロンプトでは、今回変更する箇所と変更してはならない箇所を具体的に指定する。

具体的には、変更する機能や処理部分を指定するとともに、既存の UI デザイン、操作フロー、正常に動作している機能などについては、変更対象に含めないことを指示する。

また、修正によって既存機能に不具合が生じた場合に備え、変更前の安定した状態を保持し、必要に応じて変更前の状態へ戻せるようにすることも指示に含める。

このように、共通プロンプトと個別プロンプトを使い分けることで、開発方針を維持するとともに、生成 AI による意図しない変更を抑制することを目指した。

4.2.3 プロンプトの具体例

本研究で採用した指示方針を、以下のようなプロンプトとして整理できる。

なお、以下は指示方針を文章化した代表例であり、過去に使用したプロンプトを逐語的に引用したものではない。

共通プロンプトの設定例

- 改善案を提示する場合は、原則として 3 つ程度の選択肢を示す。
- 技術的な説明では、情報の根拠となる公式資料や参考資料へのリンクを提示する。
- 制作の主目的を常に優先する。
- 既存機能や UI の操作性を不用意に変更しない。
- 不具合が発生した場合に備え、変更前の状態を維持する方法を考慮する。

個別プロンプトの指示例

Quick Actions の制作における最優先の目的は、プログラミング初学者が開発作業で困難に感じる操作を、機能の意味を理解しながら、簡単かつ安全に実行できるよう支援することである。

今回の修正では、以下の条件を必ず守ること。

【変更対象】

- 今回指定した機能・ファイル・処理部分のみを変更する。
- 指定箇所以外への変更は行わない。
- 既存機能との整合性を確認する。

【変更してはならない箇所】

- 今回指定していない機能は変更しない。
- 既存の UI デザインや操作フローを勝手に変更しない。
- 正常に動作している既存機能を削除・簡略化しない。
- 既存の内部処理を、指示なく別の方式に変更しない。
- Quick Actions の制作目的を損なう変更を行わない。

【ロールバックへの対応】

- 変更前の安定した状態を保持する。
- 修正前のファイルまたはバージョンに戻せる状態を確保する。
- 修正によって不具合が発生した場合は、必要に応じて変更前の状態へ戻す。
- 問題の原因が特定できない場合、無関係な箇所を追加修正して解決しようとしない。
- 変更前後の差分を明確にする。

【動作確認】

- 修正した機能が目的どおり動作するか確認する。
- 既存機能に影響が生じていないか確認する。
- 不具合が発生した場合は、原因と変更箇所を明示する。

【最優先事項】

機能追加やコードの変更よりも、既存機能の維持と Quick Actions 本来の制作目的を優先すること。

このような指示を用いることで、生成 AI に修正内容だけでなく、維持すべき仕様や復旧に関する条件も伝える方針とした。

4.2.4 ロールバックを考慮した開発

本開発では、新しい機能の追加だけでなく、修正によって既存機能が正常に動作しなくなる問題も経験した。

具体的には、バージョン 1.3.0、1.5.0 および 1.8.0 以降の開発において問題が発生した。

バージョン 1.3.0 では、動作の改善を目的としてプロンプトを設計したが、その結果、Quick Actions のボタン UI 自体に全文コピーの動作が付与されるという意図しない変更が発生した。これにより、通常の操作性が大きく損なわれた。

バージョン 1.5.0 では、ソフトウェアをインポートしても正常に使用できない不具合が発生した。

さらに、バージョン 1.8.0 以降では、ボタンを押しても反応しないなど、基本的な操作が実行できなくなる問題も発生した。

これらの経験から、生成 AI による変更は、指定した機能の追加や改善だけでなく、既存機能やソフトウェア全体に予期しない影響を与える可能性があると考えられる。

その対策として、本開発では、正常な動作を確認した旧バージョンを必ず手元に保管することを開発手順に組み込んだ。

新しいバージョンを制作する際には、変更前のソースコードや配布用ファイルを保持したうえで修正を行い、変更後には既存機能を含めた動作確認を実施する。

また、問題が発生した場合には、開発者自身が旧バージョンへの復元を行える状態を維持する。

実際に、開発中の問題に対するロールバックは 3 回実施しており、いずれも開発者自身が復旧作業を行った。

このことから、AI による変更を管理するためには、AI への指示だけでは不十分であり、人間自身が正常なバージョンを判断し、必要に応じて復旧できることが重要であると考えられる。

4.2.5 開発工程

本開発では、次の工程を繰り返すことで機能の追加と改善を行った。

1. 制作の主目的と変更したい内容を明確にする。
2. 生成 AI に変更対象と変更禁止箇所を提示する。

3. 変更前の正常動作が確認された旧バージョンを保管する。
4. 生成 AI から実装案や修正コードを得る。
5. 変更内容を確認したうえで、修正を反映する。
6. 実際の環境で動作を確認する。
7. 既存機能への影響や不具合の有無を確認する。
8. 問題が発生した場合は修正を取り消すか、必要に応じて旧バージョンへ戻す。

この工程により、生成 AI の出力をそのまま採用するのではなく、開発者が仕様との整合性や動作結果を確認しながら開発を進めた。

4.2.6 生成 AI によるコード生成と開発者自身による対応

本開発では、生成 AI によるコード生成を合計 14 回実施した。また、開発者自身による対応は合計 4 回であった。開発者自身による対応の内訳は、以下のとおりである。

対応内容	回数
旧バージョンへのロールバック	3 回
コードの可読性を高めるための修正	1 回
開発者自身による対応の合計	4 回
生成 AI によるコード生成	14 回

開発者自身による対応のうち 3 回は、生成 AI による変更後に問題が発生した際、正常に動作していた旧バージョンへ戻すために実施したものである。残りの 1 回は、生成されたコードの可読性を高めるために開発者自身が修正したものである。

この記録から、生成 AI によるコード生成を利用する場合でも、すべての作業を AI に任せられるわけではなく、人間による確認や対応が必要になることが分かる。

特に、本開発では 3 回のロールバックを開発者自身が実施しており、AI による修正結果を評価し、必要に応じて以前の正常な状態へ戻すことが重要であると考えられる。

なお、これらの回数は本開発における作業記録であり、生成 AI と人間の作業効率や生産性を一般的に比較するための数値ではない。

4.3 内部構成

Quick Actions は、主に拡張機能本体、Webview 画面、テンプレートデータの 3 つで構成される。

4.3.1 拡張機能本体

拡張機能本体は、VS Code API を利用してエディターの状態を取得し、各操作を実行する役割を持つ。主な処理は以下のとおりである。

- ボタン操作に対応するコマンドの登録
- アクティブエディターの取得
- 選択範囲およびファイルの編集可否の確認

- コードの編集・変換処理
- テンプレートの挿入
- Webview とのメッセージ通信

操作ごとに必要な条件を設定し、実行前に確認する構成とした。

4.3.2 Webview 画面

Webview では、HTML、CSS、JavaScript を利用して操作画面を構築した。

ボタンのクリックやテンプレートの選択などの利用者による操作は、メッセージとして拡張機能本体へ送信される。

拡張機能本体は、受け取ったメッセージに応じて処理を実行し、その結果を Webview に返す。

この仕組みにより、画面表示とエディターの編集処理を分離した構成とした。

4.3.3 テンプレートデータ

テンプレートの内容は、templates.json にまとめて管理する。

各テンプレートには固有の ID を設定し、言語、タイトル、説明、コード本文、挿入位置、必要な準備などを保持する。

Webview 側でテンプレートの一覧表示や検索を行い、選択されたテンプレート ID を拡張機能本体へ送信することで、対応するコードを取得して挿入する。

4.4 操作の安全性と不具合防止

本システムでは、誤操作によるファイル編集や意図しない処理を防ぐため、以下の確認処理を実装した。

1. アクティブエディターが存在するか確認する。
2. ファイルが読み取り専用ではないか確認する。
3. 選択範囲が必要な操作では、対象の文字列が選択されているか確認する。
4. ファイル全体に影響する空白削除などの処理では、実行前に確認を表示する。
5. ファイルの言語と異なるテンプレートを挿入する場合は、利用者に確認を求める。
6. 選択中の文字列をテンプレートで置き換える場合は、事前に確認を表示する。
7. 処理中の連続操作を抑制する。

また、コード編集やテンプレート挿入には VS Code の編集機能を使用し、編集履歴から操作を取り消せる構成とした。

4.5 セキュリティと軽量性

Webview では Content Security Policy を設定し、nonce を利用してスクリプトおよびスタイルの実行を制限した。

また、テンプレートのプレビュー表示には textContent を使用し、テンプレート本文を HTML として直接解釈させない構成とした。

テンプレートデータはローカルの JSON ファイルとして同梱しており、外部通信や追加の npm 依存パッケージを使用しない設計とした。

これらは実装上の対策であり、システム全体のセキュリティが保証されることを示すものではない。

第 5 章 動作検証および第三者評価

5.1 評価目的

本研究で開発した Quick Actions について、実装した機能や基本的な処理が設計どおりに構成されているかを確認するとともに、第三者に使用してもらい、操作性および利便性に関する意見を収集する。

本評価は、厳密な条件を設定した大規模なユーザビリティ実験ではなく、個人開発で制作したツールについて、実際の使用感や改善点を把握するための簡易的な確認として実施した。

特に、以下の観点を中心に確認した。

1. ボタンによる視覚的な操作方法は理解しやすいか。
2. VS Code 標準のショートカットキーと比較して、ボタン操作は利用しやすいか。
3. ショートカットキーを覚えていなくても必要な機能を利用できるか。
4. これまで知らなかった機能を発見し、実際に使用できるか。
5. 機能の配置や画面構成に不便な点はないか。

5.2 自動テストによる検証

開発した拡張機能について、Node.js の標準テスト機能を使用した自動テストを実施した。

テストでは、以下の項目を確認した。

検証項目	確認内容
ビュー登録	Webview および Secondary Side Bar への登録
画面内 JavaScript	構文解析
テンプレート挿入	言語および選択範囲に応じた処理
読み取り専用対応	読み取り専用ファイルへの挿入防止
言語選択	未対応言語でのテンプレート言語選択
テンプレート構成	各言語 20 種類の登録
入力箇所	Tab 移動用プレースホルダーの形式

対象となる Quick Actions 1.7.0 に含まれる自動テストを実行した結果、7 件すべてのテストが成功した。また、extension.js に対する JavaScript 構文チェックも成功した。

ただし、これらのテストには VS Code API を模擬した環境での検証が含まれている。そのため、すべての操作について、実際の VS Code 上での動作を確認した結果とは区別する必要がある。

また、この自動テストの成功はバージョン 1.7.0 を対象とした結果であり、過去のすべてのバージョンにおける問題が解消されたことを示すものではない。

5.3 第三者による使用感の確認

Quick Actions の操作性および利便性について、第三者に実際に使用してもらい、簡易的な意見収集を行った。

使用者は友人・知人を含む 4 名であり、そのうち 3 名から回答を得た。

項目	人数
使用者数	4 名
回答者数	3 名
回答率	75%

使用者には実際に拡張機能を操作してもらい、その後、口頭で使用感や改善点について意見を聞いた。

確認した内容は、以下の項目を中心とした。

- 操作方法は分かりやすかったか。
- 機能の説明は理解しやすかったか。
- VS Code 標準のショートカットキーによる操作と比較して、Quick Actions のボタン操作は利用しやすいか。
- ショートカットキーを多数覚えずに利用できる点に利便性を感じるか。
- これまで知らなかった便利な機能を利用するきっかけになるか。
- 便利だと感じた機能は何か。
- 不便だと感じた部分は何か。
- 改善してほしい部分は何か。

今回の確認は、利用者全体に対する有効性を測定する正式な調査ではなく、開発したツールの使い勝手を把握するための簡易的な評価である。

5.4 第三者評価の結果

使用者から得られた主な意見を、使いやすい点と使いにくい点に分けて整理した。

5.4.1 使いやすいと評価された点

評価項目	具体的な内容
視覚的な分かりやすさ	ボタンに機能名が表示されているため、操作方法や機能の内容を把握しやすい
ショートカットキーの記憶負担	多数のショートカットキーやキーの組み合わせを覚えなくても操作できる
操作の簡便さ	キー入力の代わりに、目的のボタンを 1 回クリックするだけで操作を実行できる
機能の発見しやすさ	これまで知らなかった便利な操作機能を、ボタン一覧から発見して利用できる
一括保存機能	複数のファイルを個別に保存する必要がなく、一度の操作でまとめて保存できる

評価項目	具体的な内容
行の上下移動	HTML などのコードの記述順序を変更する際、対象の行を簡単に上下へ移動できる

操作方法や機能の説明については、機能を視覚的に確認できる点に対して肯定的な意見が得られた。

特に、VS Code 標準のショートカットキーと比較した場合、多数のキーの組み合わせを記憶する必要がなく、画面上のボタンを 1 回クリックすることで目的の操作を実行できる点が利便性として挙げられた。

標準ショートカットキーは、操作方法を把握している利用者にとって便利な方法である一方、初学者はショートカットキーの存在やキーの組み合わせを知らなければ、機能を利用するまでに調べる必要がある。

これに対して Quick Actions では、機能名がボタンとして表示されているため、利用者は操作方法を覚えていなくても、画面上から機能を選択できる。

また、標準機能として用意されている便利な操作であっても、利用者がその存在を知らなければ活用することは難しい。Quick Actions では、ボタン一覧から機能の存在を認識し、実際に使用できる点についても肯定的な意見が得られた。

さらに、一括保存機能については、複数のファイルを同時に編集している場合に、個別に保存操作を繰り返すことなく、一度の操作でまとめて保存できる点が利便性として挙げられた。個別に保存する場合と比較して操作手順を減らせるため、複数のファイルを扱う場面で特に有用であると考えられる。一方、開いているファイル数が少ない場合には、個別保存との操作上の差が小さく、一括保存機能による利便性も限定的になる。

また、行を上下に移動する機能についても、実際のコード編集における利便性が評価された。例えば、HTML を記述する際に、ナビゲーションなどの要素の表示順序を変更したい場合、移動対象の行にカーソルを置き、上下に移動する操作を実行することで、コードの並び替えを簡単に行える。この機能により、対象のコードを別の位置に移動するための編集操作を簡略化できる点が利便性につながったと考えられる。

5.4.2 使いにくいと評価された点

評価項目	具体的な内容
機能の表示位置	目的の機能がどこにあるのか分かりにくい場合がある
機能の位置調整	利用者が使いやすい位置へ調整する操作を、さらに簡単にしてほしい
選択操作	1 行単位で簡単に選択できる機能が欲しい
ボタンの配置	2 列配置ではボタンが縦方向に並び、画面全体が縦長になって使いにくい

機能の表示位置については、目的の機能を見つけにくいという意見が得られた。

また、機能の配置を利用者自身が簡単に調整できるようにしてほしいという改善要望もあった。

編集操作については、1 行単位で選択できる機能を追加してほしいという意見が得られた。現在の操作方法では編集対象の指定に手間がかかる場合があり、カーソル位置の行を簡単に操作対象とできる仕組みが求められている。

さらに、ボタンの 2 列配置については、横方向のスペースを活用できる一方、ボタンが縦方向に並ぶことによって画面全体が長くなり、使いにくいという意見があった。

このことから、ボタンを複数列に配置するだけでは必ずしも操作性の向上につながらず、画面内の表示領域やボタン数とのバランスを考慮する必要があると考えられる。

5.5 評価結果の整理

今回の使用感の確認では、Quick Actions の利点として、ショートカットキーの記憶負担を軽減できること、機能を視覚的に把握できること、これまで知らなかった便利な操作機能を発見して利用できることが挙げられた。

特に、VS Code 標準のショートカットキーと比較して、ボタンを 1 回クリックするだけで目的の操作を実行できる点は、利用者にとって使いやすい特徴であると考えられる。

また、標準機能の存在やショートカットキーを知らない場合でも、Quick Actions のボタン一覧から機能を確認できるため、新しい操作方法を知るきっかけになる可能性がある。

一括保存機能については、複数のファイルを個別に保存する操作を一度にまとめられる点が利便性として挙げられた。特に、複数のファイルを同時に編集する場面では、保存操作の手順を減らせる。一方、編集対象が少ない場合には、個別に保存する場合との操作上の差が小さくなることから、利用状況によって利便性が異なると考えられる。

さらに、行の上下移動機能についても評価された。HTML のナビゲーションなど、要素の記述順序を変更する場面では、対象の行を上下に移動するコマンドをすぐに実行できるため、コードの並び替えを簡単に行える。このように、日常的なコード編集で発生する細かな作業をボタンから実行できることも、本ツールの利点の一つである。

一方で、機能の表示位置が分かりにくいことや、2 列配置によって画面が縦方向に長くなること、1 行単位で選択できる機能が欲しいことなどの改善意見も確認された。

今回得られた意見から、ボタンによる操作の簡便さだけでなく、目的の機能を見つけやすい配置や、編集対象を指定しやすい操作方法も重要であると考えられる。

なお、今回の使用感の確認では、4 名が実際に使用し、そのうち 3 名から回答を得た。項目別の回答人数や割合を集計したものではないため、意見の内容を中心に整理している。また、ショートカットキーとの比較は使用者の主観的な意見に基づくものであり、操作時間を測定した定量的な比較ではない。

第 6 章 考察

6.1 視覚的な情報提示および標準ショートカットとの比較について

第三者による使用感の確認では、操作方法や機能の説明について、視覚的な情報提示が理解しやすさにつながったとの意見が得られた。

この結果から、本研究で採用したボタン形式の操作画面は、利用者が機能を認識するうえで役立つ可能性があると考えられる。

特に、各操作を文字として表示し、分類ごとに整理する設計は、利用者がコマンド名やキーボードショートカットを記憶していなくても、目的の機能を画面上から認識できるようにすることを意図したものである。

VS Code の標準ショートカットキーは、操作方法を把握している利用者にとって効率的な操作手段である。一方、操作によってはキーの組み合わせを記憶する必要があり、初学者にとっては操作方法を習得する負担になる可能性がある。

また、便利な標準機能が存在していても、その機能やショートカットキーを知らなければ、利用者が自力で発見することは難しい場合がある。

これに対して Quick Actions では、操作内容をボタンとして表示することで、利用者がショートカットキーを覚えていなくても、ボタンをクリックするだけで対応する操作を実行できる構成とした。

さらに、画面上に機能名が表示されるため、これまで知らなかった機能の存在を認識し、実際の作業で利用するきっかけになることも期待できる。

今回の使用感の確認でも、ショートカットキーを多数覚える必要がない点や、知らなかった機能をボタンから簡単に利用できる点について肯定的な意見が得られた。

このことから、Quick Actions の利点は、単純に操作をボタン化することだけではなく、標準機能の発見しやすさと利用のしやすさを補助する点にもあると考えられる。

この設計は、Shneiderman が論じた直接操作の考え方や、Nielsen が示した「記憶よりも認識を重視する」というユーザビリティの原則と関連している[1][2]。

ただし、標準ショートカットキーと比較して実際にどの程度操作時間が短縮されたかについては、今回測定していない。そのため、Quick Actions がすべての操作において標準ショートカットよりも効率的であると結論づけることはできない。

本研究では、ショートカットキーの習得を補助する別の操作手段として、ボタンによる視覚的な操作方法を提供した点を評価する。

6.2 一括保存機能および行移動機能の利便性について

第三者評価では、一括保存機能および行の上下移動機能について、開発作業における利便性が確認された。

一括保存機能は、複数のファイルを編集している場合に、個別の保存操作を繰り返すことなく、一度の操作でまとめて保存できる点に特徴がある。個別に保存する場合と比較して操作手順を減らせるため、複数のファイルを同時に編集する場面で特に利便性が高いと考えられる。

一方、開いているファイルや編集対象が少ない場合には、個別保存との操作上の差が小さくなるため、その効果は限定的である。このことから、一括保存機能の利便性は、利用者が同時に扱うファイル数や作業状況によって異なると考えられる。

また、行の上下移動機能は、コードの記述順序を変更する場面で役立つ。例えば、HTML においてナビゲーションなどの表示順序を変更する際、対象の行にカーソルを置き、上下に移動する操作をボタンから実行できる。これにより、対象のコードを別の位置へ移動するための編集操作を簡略化できる。

特に、HTML の構造を調整する場合には、要素の配置や記述順序を変更する作業が発生する。行の上下移動機能を利用することで、対象となる行を簡単に移動でき、細かなコード編集を進めやすくなる。

これらの機能は、複雑な処理を自動化するものではないが、開発中に繰り返し発生する操作を簡単に実行できる点に特徴がある。

このことから、初学者向けの開発支援ツールでは、専門的な機能を追加するだけでなく、保存やコードの並び替えなど、日常的な開発作業で発生する操作を簡単に実行できることも重要であると考えられる。

6.3 既存拡張機能との差別化について

既存の VS Code 標準機能や拡張機能との比較から、Quick Actions の特徴は、個々の機能が新しいことではなく、初学者向けの操作環境として複数の機能を一つの画面に統合した点にあると考えられる。

例えば、VS Code 標準ショートカットはキーボードによる操作を効率化し、標準スニペットはコードの挿入を補助する。また、Multi-command は複数のコマンドを連続実行でき、Quick Command Toolbar は独自のボタンからコマンドを実行できる。

これらの機能は、それぞれ異なる目的や利用方法を持つ。

一方、Quick Actions では、基本的な編集操作、コード整理・変換、複数言語のコードテンプレートを同じ画面にまとめている。

特に、初学者にとっては、必要な機能を探すために複数の拡張機能の使い方を覚える必要がなく、同じ操作パネルから目的の機能を選択できることが利点になると考えられる。

また、テンプレートには説明やプレビューを設けているため、単にコードを挿入するだけでなく、利用者が挿入する内容を確認できる構成とした。

ただし、既存拡張機能にもボタン操作やコマンド実行を簡略化する仕組みが存在するため、Quick Actions が独自の操作方式を実現したと主張するものではない。

本研究では、初学者の利用を意識した機能の組み合わせと、操作・コード整理・テンプレート挿入を一つの画面で扱える構成を、設計上の特徴として位置づける。

6.4 機能配置の課題について

機能の表示位置が分かりにくいという意見や、2 列配置によって画面が縦方向に長くなり、使いにくいという意見が得られた。

本研究では、Explorer と Quick Actions を同時に利用できるようにするため、操作パネルを右側の Secondary Side Bar に配置した。

この変更によって、左側の Explorer と表示領域が競合する問題への対応を図った。

しかし、表示領域を左右に分離することと、Quick Actions 内の機能を探しやすくすることは別の問題である。

また、2 列配置は横方向のスペースを活用できる一方、ボタン数が多くなると縦方向の表示領域が大きくなる。実際に使用した利用者からも、画面が縦長になり、使いにくいという意見が得られた。

このことから、開発者が意図した配置と、利用者が直感的に理解できる配置が必ずしも一致しない可能性がある。

そのため、今後はボタンの配置や機能の分類、表示領域の使い方を見直し、利用者が目的の機能を発見しやすい画面構成を検討する必要がある。

6.5 編集操作の柔軟性について

1 行単位で選択できる機能に対する要望は、編集対象を指定する操作を簡単にしたいという利用者の意見を示している。

本システムでは、行末の空白削除、空行の削除、インデントの追加・削除などの編集機能を実装しているが、利用者が編集対象を指定する方法については、さらに検討する余地がある。

今後は、カーソル位置の行だけを簡単に選択できる機能などを検討する必要がある。

6.6 バイブコーディングを用いた開発について

本研究では、生成 AI に対して制作の主目的や実装要件を提示し、自然言語による指示を通じてコード生成や修正を行った。

特に、変更対象と変更禁止箇所を明示し、修正によって既存機能に影響が生じた場合に備えて、ロールバックが可能な状態を維持するよう指示した点が特徴である。

このような開発方法では、開発者が実装内容をすべて手作業で記述する場合とは異なり、機能要件の整理や生成 AI への指示内容が開発工程において重要となる。

本開発では、生成 AI によるコード生成を 14 回実施し、開発者自身による対応を 4 回行った。そのうち 3 回は旧バージョンへのロールバックであり、残り 1 回はコードの可読性を高めるための修正であった。

この記録は、生成 AI を利用した開発であっても、人間による確認や対応が必要になることを示す一つの開発事例である。

一方、生成 AI が提示するコードには、要求と異なる処理や予期しない変更が含まれる可能性がある。そのため、変更範囲を限定する指示に加えて、コードの確認、テスト、動作検証を行う必要がある。

また、今回の開発では、バージョン 1.3.0 における意図しない全文コピー動作の追加、バージョン 1.5.0 におけるインポート後の利用不能、バージョン 1.8.0 以降におけるボタン操作の不具合を経験した。

これらの事例から、プロンプトによって変更範囲を指定しても、不具合や意図しない変更を完全に防止できるわけではないことが分かった。

そのため、変更前の正常動作が確認された旧バージョンを手元に保管し、問題が発生した場合には開発者自身が復元できる状態を維持する必要がある。

特に、旧バージョンへのロールバックは、AI にすべてを任せるのではなく、開発者自身が実施できることが重要である。AI が生成したコードの内容を確認し、以前の正常な状態と比較したうえで、必要に応じて自ら復元する対応が求められる。

さらに、継続的に適用する共通プロンプトと、個別の修正範囲を指定する個別プロンプトを使い分けることで、開発全体の方針と個別作業の条件を管理できると考えられる。

生成 AI の利用に関する筆者の所感

筆者は、生成 AI がソフトウェア開発における作業を支援する重要なツールになりつつあると考えている。

しかし、生成 AI は便利である一方、適切に利用するためには注意も必要である。指示内容によって出力が変化し、意図しない変更や不具合が生じる可能性もある。

今後、生成 AI を利用する機会が増える中で、利用者が生成 AI の出力や判断に過度に依存し、自らの目的や判断基準を見失わないことが重要であると考える。

そのためには、利用者自身が継続的に適用する共通プロンプトを設定し、個別の作業では変更範囲を明確に指示することが必要である。

さらに、AI が生成したコードや提示した情報を無条件に受け入れるのではなく、内容の確認や動作検証を利用者自身が行うことが求められる。

不具合が発生した際にも、AI に復旧をすべて任せるのではなく、自分自身で旧バージョンへの復元を行える状態を維持する必要がある。

生成 AI を利用すること自体を目的とするのではなく、作業を支援する道具として適切に活用し、最終的な判断と責任を人間が持つ姿勢が、今後の開発において重要になると考える。

6.7 本研究の限界

本研究には、いくつかの制約がある。

第一に、使用者は 4 名、回答者は 3 名と少人数であり、友人・知人を中心とした使用感の確認である。そのため、得られた意見をすべてのプログラミング初学者に一般化することは難しい。

第二に、評価は口頭での質問を中心としており、操作時間や操作ミスの回数などの定量的な測定は実施していない。

特に、VS Code 標準のショートカットキーと Quick Actions の比較についても、使用者の主観的な意見を収集したものであり、操作時間や作業効率を数値で比較したものではない。

第三に、既存の VS Code 標準操作との定量的な比較を行っていないため、Quick Actions によって実際の作業時間や操作負担がどの程度変化したかは明らかになっていない。

第四に、既存拡張機能との比較は公開されている機能説明に基づくものであり、実際の使用環境における操作性や性能については検証していない。

第五に、本開発では生成 AI によるコード生成・修正を利用している。そのため、生成 AI が提示したコードの内容や修正案が開発工程に影響している。使用した AI モデルやバージョン、主要なプロンプト、修正履歴などを記録することで、開発工程の再現性をさらに高める必要がある。

第六に、本開発で経験した不具合について、すべての発生原因を技術的に特定し、共通する原因を実験によって検証したわけではない。したがって、今回の問題事例から、生成 AI 全般の不具合発生率や特定のプロンプト設計の有効性を断定することはできない。

第七に、今回の使用感の確認では、各評価項目について回答者数を個別に集計していない。そのため、特定の意見が回答者の何名から得られたかという割合を算出することはできない。今後、評価内容をより詳細に記録する場合には、各質問項目に対する回答数や具体的な回答内容を整理する必要がある。

以上の制約から、本研究で得られた結果は、開発した拡張機能に対する初期的な使用感の確認と、個人開発における実践経験として位置づける必要がある。

第 7 章 結論および今後の課題

7.1 結論

本研究では、プログラミング初学者の開発操作を支援することを目的として、Visual Studio Code 拡張機能 Quick Actions の設計および開発を行った。

本拡張機能では、基本的な編集操作や不要な空白の削除、文字変換などのコード整理・変換機能をボタンとして配置し、GUI から操作できる仕組みを実装した。

また、C++、Python、Java の 3 言語に対応した合計 60 種類のコードテンプレートを収録し、構文やアルゴリズムを簡単な操作で挿入できる構成とした。

本ツールは、開発者自身の作業効率を向上させる個人用ツールとして制作を開始したが、VS Code の操作やプログラミングに慣れていない初学者でも利用できることを意識して設計した。

開発には、自然言語による指示を通じて生成 AI にコード生成・修正を依頼するバイブコーディングを採用した。開発者が制作の主目的を明確に提示し、変更対象、変更禁止箇所、ロールバックへの対応を指定しながら、実装と動作確認を繰り返した。

開発過程では、生成 AI によるコード生成を 14 回実施し、開発者自身による対応を 4 回行った。その内訳は、旧バージョンへのロールバックが 3 回、コードの可読性を高めるための修正が 1 回である。

また、バージョン 1.3.0 における意図しない全文コピー動作の追加、バージョン 1.5.0 におけるインポート後の利用不能、バージョン 1.8.0 以降におけるボタン操作の不具合などを経験した。

これらの問題への対応では、開発者自身が旧バージョンへのロールバックを実施した。この経験から、生成 AI への指示管理だけでなく、変更前の正常なバージョンを手元に保管し、必要に応じて開発者自身が復元できる状態を維持することが重要であると考えた。

また、継続的に適用する共通プロンプトと、個別の変更範囲を指定する個別プロンプトを使い分ける方法についても整理した。

開発後には自動テストによる検証を実施し、対象バージョン 1.7.0 の 7 件すべてのテストが成功した。

さらに、使用者 4 名のうち 3 名から、簡易的な使用感に関する回答を得た。その結果、視覚的な情報提示による操作方法の理解しやすさや、一括保存機能の利便性について肯定的な意見が得られた。

特に、VS Code 標準のショートカットキーと比較して、多数のキーの組み合わせを覚えなくても、ボタンを 1 回クリックすることで目的の操作を実行できる点や、これまで知らなかった便利な機能を発見して利用できる点についても肯定的な意見が得られた。

また、一括保存機能については、複数のファイルを個別に保存する操作を一度にまとめられる点が利便性として挙げられた。特に、複数のファイルを同時に編集する場面では、保存操作の手順を減らせる一方、編集対象が少ない場合には、その効果が限定的になると考えられる。

さらに、行の上下移動機能についても、HTML のナビゲーションなどの記述順序を変更する際に、対象の行を簡単に移動できる点が利便性として挙げられた。

一方で、機能の表示位置が分かりにくいこと、2 列配置による画面の縦長化、1 行単位で選択できる機能への要望など、改善すべき点も確認された。

既存拡張機能との比較では、基本操作、コード整理・変換、複数言語のコードテンプレートを一つの画面に統合したことを、Quick Actions の設計上の特徴として整理した。

以上から、本研究で開発した拡張機能について、視覚的な操作画面やショートカットキーの習得を必要としない操作方法に対する肯定的な意見が得られた。また、一括保存や行の上下移動といった機能についても、実際の開発作業における操作手順を簡略化できる点が評価された。一方、機能の配置や選択操作については改善の余地があることが確認された。

ただし、今回の使用感の確認は少人数によるものであり、プログラミング初学者全体に対する有効性を実証したものではない。

また、本研究を通じて、生成 AI を利用した開発では、AI に作業を任せるだけでなく、利用者自身が指示を管理し、出力内容を確認し、問題が発生した際には自ら復旧できることが重要であると考えた。

7.2 今後の課題

今後の課題として、以下の点が挙げられる。

1. ユーザーインターフェースの改善

機能の表示位置や分類を見直し、利用者が目的の操作を発見しやすい画面構成を検討する。また、2 列配置によって画面が縦方向に長くなる問題についても、表示領域とのバランスを考慮した改善を検討する。

2. 編集操作の拡張

1 行単位での選択機能など、利用者から得られた要望を基に編集操作を改善する。

3. 評価対象者の拡大

より多くの利用者に操作してもらい、プログラミング経験の違いによる使用感の差を検討する。

4. 評価方法および記録内容の充実

本研究では、評価対象者数や実施環境などの制約から、操作時間や操作ミスの回数を用いた定量的な比較は実施していない。また、第三者評価についても、各評価項目の回答人数や割合を個別に集計していない。そのため、今後同様の使用感の確認を行う場合には、各質問に対する回答内容や具体的な使用場面をより詳細に記録し、機能ごとの利便性や改善点を整理できるようにする必要がある。

5. 生成 AI を利用した開発工程の記録

使用した生成 AI のモデルやバージョン、主要な指示内容、生成コードに対する修正内容などを記録し、開発工程の再現性や保守性について検討する。

6. プロンプト管理とバージョン管理の継続

共通プロンプトと個別プロンプトの使い分けを継続し、変更対象と変更禁止箇所を明確にする。また、新しいバージョンの制作前には、正常動作が確認された旧バージョンを必ず保管し、必要に応じて開発者自身で復元できる状態を維持する。

これらの改善と追加検証を通じて、開発環境の操作に慣れていない利用者が、より簡単に開発作業を進められる支援ツールを目指す。

生成 AI の利用に関する開示

本研究のソフトウェア開発では、生成 AI サービスである ChatGPT を利用した。主な用途は、自然言語による機能要件の提示に基づくソースコードの生成、既存コードの修正案の作成、エラー原因の調査支援、および実装方法の検討である。

本開発では、いわゆるバイブコーディングを採用し、生成 AI との対話を通じて機能の実装や改良を進めた。ただし、制作の主目的、機能要件、変更対象、変更禁止箇所、および採用する変更内容については開発者が判断した。

また、本稿の構成、文章の草案作成および表現の修正にも ChatGPT を利用した。研究対象の仕様、第三者による使用感の回答、テスト結果および参考文献については、提出前に著者が事実関係を確認する。

生成 AI は本研究の著者ではなく、研究内容および最終的な提出物に対する責任は著者が負う。

生成 AI サービス：ChatGPT

使用モデル・バージョン：GPT-6

参考文献

[1] Ben Shneiderman, “Direct Manipulation: A Step Beyond Programming Languages,” IEEE Computer, Vol.16, No.8, pp.57–69, 1983. https://doi.org/10.1109/MC.1983.1654471

[2] Jakob Nielsen, “Enhancing the Explanatory Power of Usability Heuristics,” Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI ’94), pp.152–158, 1994. https://doi.org/10.1145/191666.191729

[3] Microsoft, “Views | Visual Studio Code Extension API.” https://code.visualstudio.com/api/ux-guidelines/views

[4] Microsoft, “Webview API | Visual Studio Code Extension API.” https://code.visualstudio.com/api/extension-guides/webview

[5] Shalini Harkar, “What is vibe coding?” IBM Think, 2025 年 4 月 8 日公開、2026 年 10 月 28 日更新. https://www.ibm.com/think/topics/vibe-coding

[6] 著者, “VS Code Quick Actions Version 1.7.0,” ソフトウェアおよび開発資料, 2026.

[7] Microsoft, “Keyboard shortcuts for Visual Studio Code.” https://code.visualstudio.com/docs/configure/keybindings

[8] Microsoft, “Snippets in Visual Studio Code.” https://code.visualstudio.com/docs/editing/userdefinedsnippets

[9] Ryuta46, “Multi-command,” Visual Studio Marketplace. https://marketplace.visualstudio.com/items?itemName=ryuta46.multi-command

[10] Chirag Patel, “Quick Command Toolbar,” Visual Studio Marketplace. https://marketplace.visualstudio.com/items?itemName=PatelChirag.quick-command-toolbar

最終閲覧日：2026 年 11 月 11 日